JAVA PROGRAMMING – ASSIGNMENT 5 (CO5)

Data Persistence Engineering – Files and Databases

Name: J.Subhaashree
Register Number: 192424008
Platform: Google Colab, Java 17, MySQL

Assignment Report

This report follows the structure and terminology of the submitted Assignment 5 notebook. Part A documents persistence design and safety principles. Part B records the Java implementations and the experiment outputs saved in the notebook.

PART A – PERSISTENCE DESIGN & PSEUDOCODE

A1 – Design a File Format

Text layout: passId, regNo, name, routeNo, boardingStop, validityDate, amountPaid, one UTF-8 record per line. Use a CSV-aware encoder/parser to quote commas and double embedded quotes. Binary layout uses DataOutputStream writeUTF/writeInt/writeDouble in a fixed order; read with matching methods. ADD appends one validated encoded record. FIND scans records and returns the matching pass ID or NOT FOUND. LIST BY ROUTE scans and filters by route number.

A2 – Don't Read What You Don't Need

50 MiB = 52,428,800 bytes. A byte-at-a-time loop can make about 52.4 million read calls. An 8 KiB buffer reduces refills to about 6,400 while the scan remains O(n). Buffering may offer little benefit when data is already block-buffered, in memory, or CPU-bound. readAllLines uses O(file size) memory plus per-line object overhead; streaming uses O(buffer size) extra memory.

A3 – Discover the Data-Access Pattern

A DAO centralizes connection use, SQL execution, parameter binding, ResultSet mapping, exception handling and resource closure. Generic interface: Dao with insert, findById, findAll, update and delete. StudentDao implements Dao. Mapping: id → student_id (PRIMARY KEY), registerNumber → reg_no (UNIQUE), name → student_name, department → department, year → study_year.

A4 – What Should Be Serialized?

Mark live JDBC Connection transient. Do not serialize plaintext passwords. Hostel, Room and Resident should implement Serializable; serializing Hostel traverses its reachable object graph. Adding phoneNumber normally gives an older stream null for the new reference field if serialization compatibility is retained. Explicit serialVersionUID and migration/defaulting logic are recommended. CSV/JSON are portable and easier to evolve; Java serialization is convenient for object graphs but Java-specific and version-sensitive.

A5 – Correctness and Safety

Transfer ■500 in one transaction: disable auto-commit, lock/read both wallet rows, verify balance, debit and credit, commit; rollback on failure and restore auto-commit. Concatenating input into SQL enables injection. PreparedStatement uses placeholders and binds values separately. Store salted password hashes in real systems. Auto-commit can leave a debit committed without its matching credit if the second operation fails; a transaction ensures atomicity.

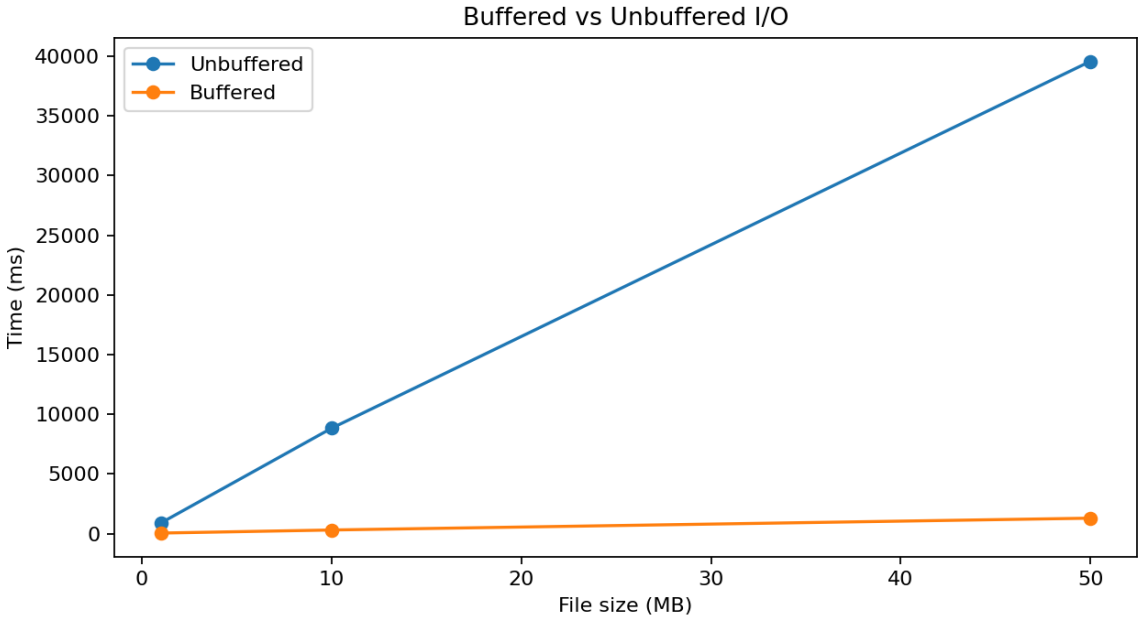
PART B – IMPLEMENTATION & EXPERIMENTAL INVESTIGATION

B1 – File Store and Buffered I/O

PassStore.java implements add() and find(), skips malformed records, and includes buffered and unbuffered line-count methods. The saved notebook output confirms that a valid pass was found, while damaged and missing IDs returned null and the scan continued.

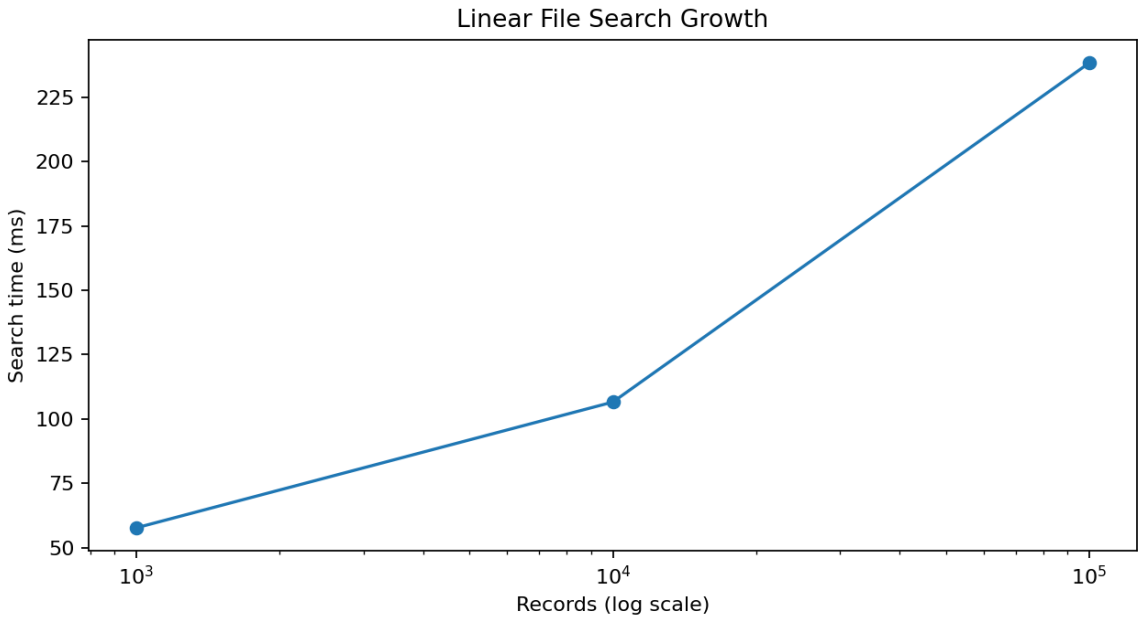
B1.3 – Buffer benchmark (recorded notebook measurements)

File size	Unbuffered (ms)	Buffered (ms)
1 MB	891.958	45.980
10 MB	8,831.046	303.326
50 MB	39,576.871	1,294.046



B1.4 – Search growth

Records	Search time (ms)
1,000	57.699
10,000	106.557
100,000	238.347



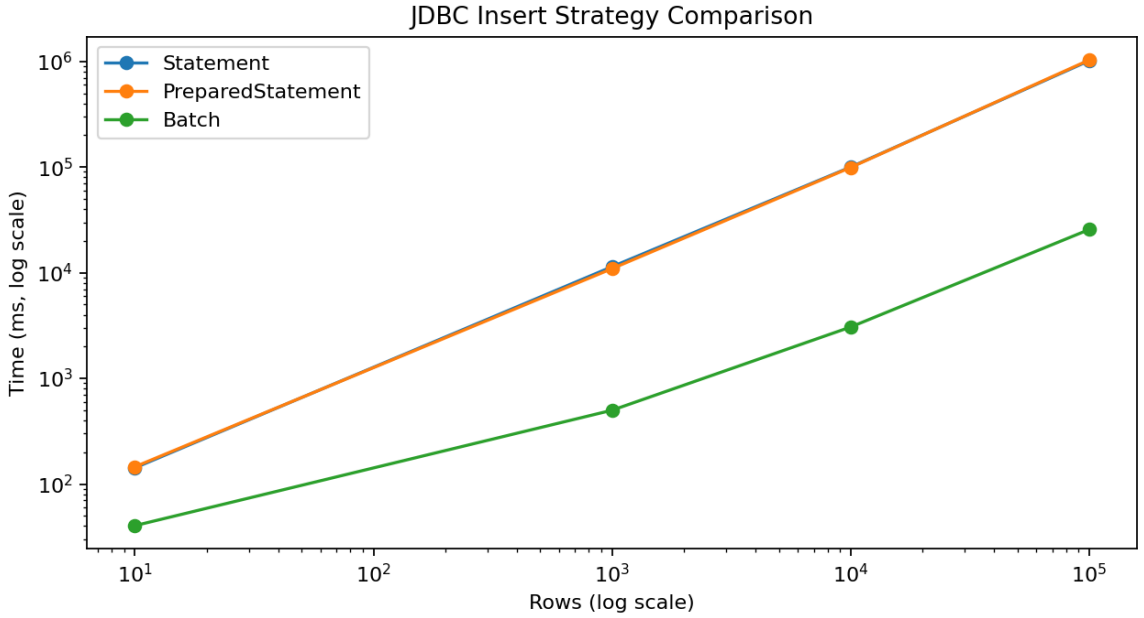
B2 – Adversarial Input

LoginSecurity.java demonstrates the difference between an unsafe concatenated login query and a parameterized PreparedStatement. Recorded output: Unsafe accepted? true; PreparedStatement accepted? false. This demonstrates why parameter binding is required for untrusted input.

B3 – Statement vs PreparedStatement vs Batch

JdbcBenchmarks.java compares Statement, PreparedStatement and batch insertion for 10, 1,000, 10,000 and 100,000 rows. It checks ResultSetMetaData and deliberately forces a failure during a transaction. The recorded final row count after rollback is zero.

Rows	Statement ms	Prepared ms	Batch ms
10	142.296	145.781	40.388
1,000	11,402.135	10,936.767	499.540
10,000	100,108.332	99,137.516	3,063.774
100,000	1,015,529.078	1,032,297.408	25,659.973



B4 – Migration and Database Metadata

MigrationAndMetadata.java reads the sample CSV, rejects malformed rows, inserts valid records in one batch transaction, and prints database/driver metadata and table names. Recorded output: lines 3 and 4 rejected for invalid field count and invalid route number; two valid records loaded. Database: MySQL 8.0.46; driver: MySQL Connector/J 9.5.0; tables: bus_pass and users_test.

B4.3 – Production decision

For 12 campuses, 100× current pass volume and concurrent counters, a relational database is the recommended shared store. It supports indexed search, concurrent clients, transactions, constraints, access control and centralized backups. Text files remain useful for import/export but are weak as a shared concurrent store. Java serialization is Java-specific, difficult to query and difficult to coordinate among writers.

RESULT SUMMARY

The recorded experiments show substantial improvement from buffered file reading and batch database insertion for the tested workloads. Parameterized SQL rejects the injection payload that succeeds against the unsafe concatenated query. Transaction rollback leaves zero rows after the forced failure, and migration loads only valid records. Timings are environment-dependent and may vary when rerun.

APPENDIX – SOURCE FILES

Complete Java source files are provided separately in the src/ folder. The original Colab notebook, including its saved code cells and outputs, is included at the project root.